

## Scope, Related Work, and Positioning

## Scope, Related Work, and Positioning I

This section situates the exemplar scientifically and states explicit boundaries. The goal is not to compete with monographs on nonlinear programming [nosedal2006numerical; bertsekas1999nonlinear], but to show how a minimal, test-backed optimization story fits the template's reproducibility and rendering stack [peng2011reproducible].

# Classical gradient methods I

Smooth unconstrained minimization via first-order updates has a long lineage, from Cauchy's early descent perspective [cauchy1847methode] to modern treatments of convex problems [boyd2004convex] and accelerated gradient schemes [nesterov2013gradient]. Polyak's classical discussion of gradient convergence factors remains relevant when interpreting empirical iteration counts [polyak1964some]. The present manuscript restricts attention to **fixed-step** gradient descent on a **convex quadratic**, where rates reduce to scalar linear recurrences in the error ([sec:results], [eq:scalar\_linear\_update]).

## Adaptive and stochastic extensions I

Practical machine-learning optimizers (e.g., Adam [kingma2014adam]) introduce momentum, adaptive preconditioning, or noise from minibatching. Those methods are **out of scope** for `template_code_project`: the exemplar deliberately keeps the algorithm minimal so that failures (divergent  $\alpha$ , iteration caps) are interpretable without confounding from stochastic sampling or line-search logic.

## What this project proves about the template I

The scientific claims through [ @sec:introduction ], [ @sec:methodology ], and [ @sec:results ] are standard textbook material. The **non-standard** contribution is procedural: configuration in `manuscript/config.yaml` drives `run_convergence_experiment()`, figures, CSV exports, and `{{RESULT_*}}` substitution (`scripts/z_generate_manuscript_variables.py`) so that PDF, HTML, and validation logs refer to the same numbers. That pattern is what downstream projects should copy—whether the domain is optimization, differential equations, or Bayesian workflows.

# Explicit limitations I

1. **Dimensionality:** Default experiments emphasize  $d = 1$  with  $A = I$  for transparent plotting; the benchmark figure explores  $d > 1$  only with identity Hessians, so no ill-conditioning effects appear.
2. **Step-size policy:** Only constant  $\alpha$  is implemented in `src/optimizer.py`; there is no Wolfe or Armijo backtracking.
3. **Global optimization:** Convexity is assumed; no basin-hopping or restarts are studied.
4. **Numerical model:** Double-precision floating point only; no interval or arbitrary-precision analysis.

These limitations are intentional: they narrow the failure surface so that infrastructure concerns—tests, logging, figure registration, and PDF cross-references—remain visible rather than buried under algorithmic complexity.